

Single Liposome Curvature Assay

Image Analysis Protocol

From raw microscopy images to curvature-sorting plots

Version 2.1

For users with no prior coding experience

Who this is for. This document is for wet-lab users running the SLiC analysis pipeline for the first time, with no prior Python or command-line experience. Every command is shown for both Mac/Linux and Windows PowerShell. If you are already comfortable with Python and the terminal, the `README.md` file in the repository is a more concise reference.

Contents

1	Overview	4
2	Quick Primer: Using the Terminal	4
3	Prerequisites	5
3.1	Python	5
3.2	uv (Package Manager)	5
3.3	Visual Studio Build Tools (for the native detection backend)	5
3.4	MATLAB and CMEanalysis (reference/fallback only)	6
3.5	DLS data	7
4	Download the Code and Set Up	7
4.1	Get the repository	7
4.2	Open a terminal in the repo folder	7
4.3	Create a virtual environment	8
4.4	Activate the virtual environment	8
4.5	Install the Python packages	8
4.6	Verify everything works	8
4.7	Build the native detection backend (recommended)	8
5	Folder Structure: Where to Put Your Files	9
6	Step 1: Prepare Images	10
7	Step 2: Run Spot Detection	12
8	Step 3: Analyze Detection Output	15
9	Step 4: DLS Calibration	17
10	Step 5: Plot Curvature Sorting	19
11	Step 6 (Optional): Plot Histograms	20
12	Step 7 (Optional): Normalized Overlay Across Experiments	21
13	Step 8 (Optional): DLS Distribution Plot	22
14	Step 9 (Optional): DLS vs Fluorescence Comparison	22
15	Quick Reference: Every-Time Checklist	23
16	Troubleshooting	23
16.1	Setup errors	23
16.2	File and path errors	24
16.3	Pipeline-stage errors	24
16.4	Plot quality issues	25
16.5	General	25

17 Common Pitfalls	25
18 Further Reading	26

1 Overview

This protocol takes you from raw two-channel fluorescence microscopy images all the way to a plot of **protein surface density vs. liposome diameter** (curvature). It is the computational half of the single-liposome curvature (SLiC) assay.

What the assay measures

Each liposome in your image appears as a sub-diffraction-limit fluorescent spot. The **lipid channel amplitude** is proportional to the liposome’s surface area, so its square root is proportional to diameter. The **protein channel amplitude** tells you how much protein is bound. By calibrating fluorescence amplitude against an independent dynamic light scattering (DLS) measurement of liposome size, you get a conversion factor from amplitude to physical diameter. The protein surface density is then

$$\rho_{\text{protein}} = \frac{A_{\text{protein}}}{\pi D^2}$$

which you plot versus diameter. A curvature-sensing protein shows ρ_{protein} rising at small diameters (high curvature).

The four steps

1. **Prepare images** — split microscope TIFFs into channels
2. **Detect spots** — fit Gaussians to each spot, with the native `cme_detect` program included in the repository (recommended; no MATLAB needed) or with the original MATLAB CMEanalysis code (reference/fallback)
3. **Analyze detection output** — filter puncta, export amplitudes
4. **Calibrate & plot** — DLS calibration, then make figures

2 Quick Primer: Using the Terminal

Every step in this protocol involves typing commands into a terminal (also called a *command line*). Here’s how to open one:

- **macOS:** Open Spotlight (`Cmd + Space`), type `Terminal`, press Enter.
- **Windows:** Press the Windows key, type `PowerShell`, click **Windows PowerShell**.

You’ll see a window with a blinking cursor. This is where you type (or paste) all the commands in this protocol. A few things to know:

- `cd` means “*change directory*.” It’s how you navigate to a folder. For example, `cd Desktop` moves you into your Desktop folder.
- You can paste commands with `Cmd+V` (Mac) or `Ctrl+V` / right-click (Windows). You don’t have to type them out.
- Press **Enter** after each command to run it.
- If something goes wrong, read the error message — it usually tells you what’s missing. The Troubleshooting section at the end covers common errors.

Tip. If a command spans multiple lines with a backslash `\` (Mac) or backtick ``` (Windows) at the end of each line, that’s just one command split across lines for readability. Paste the whole block at once — the shell understands the continuation.

3 Prerequisites

Before you start, you need these installed on your computer.

3.1 Python

Check if you already have Python by opening a terminal and typing:

```
python --version
```

If you see something like `Python 3.11.5`, you’re good. If you get an error or see `Python 2.x`, install Python 3:

- **macOS:** Go to python.org/downloads and download the latest Python 3. Run the installer.
- **Windows:** Same website, same download. **Important:** during installation, check the box that says “*Add python.exe to PATH*” — this is on the very first screen of the installer.

Note. On some Macs you may need to type `python3` instead of `python`. If `python` doesn’t work, try `python3`. If `python3` works, use `python3` everywhere this protocol says `python`.

3.2 uv (Package Manager)

`uv` is a fast Python package manager that replaces `pip`. It keeps dependencies organized and avoids version conflicts.

macOS / Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows PowerShell:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

After installing, **close and reopen your terminal** so it picks up the new command. Then verify it works:

```
uv --version
```

You should see a version number like `uv 0.6.x`. If you get “command not found,” close and reopen the terminal again.

3.3 Visual Studio Build Tools (for the native detection backend)

The recommended detection step (Step 2, Option A) uses a small C++ program, `cme_detect`, that ships with the repository and is compiled once on your computer. On Windows this needs the free

Microsoft C++ build tools:

1. Download **Build Tools for Visual Studio 2022** from visualstudio.microsoft.com/downloads (under “Tools for Visual Studio”). If you already have Visual Studio 2022 installed, use its installer instead.
2. In the installer select the “**Desktop development with C++**” workload. This includes the compiler and **CMake**. Install, and reboot if asked.
3. Open a *new* PowerShell window and check that `cmake --version` prints a version. If it says “not recognized,” use the **Developer PowerShell for VS 2022** entry in the Start menu instead, which has `cmake` on its path.

On macOS/Linux you need a C++17 compiler, CMake, and the GSL and libtiff development packages (e.g. `brew install cmake gsl libtiff`). Note that the native backend has so far only been regression-validated on Windows. If you cannot install build tools at all, use the MATLAB route instead (next subsection).

3.4 MATLAB and CMEanalysis (reference/fallback only)

Optional. MATLAB is only needed if you choose the MATLAB route in Step 2 (Option B). The recommended native backend does not use MATLAB at all — if you will use it, skip this subsection.

For the MATLAB route you need MATLAB installed with the CMEanalysis code from the Danuser Lab. If MATLAB is already installed on your computer, skip to step 3.

1. **Download MATLAB.** Look for your institution’s instructions. The software should be free for students and faculty.

Important: when downloading, you will be prompted to select various toolboxes. These can be installed later, but some are needed for CMEanalysis. Select all of the following:

- Curve Fitting Toolbox
 - Image Processing Toolbox
 - Optimization Toolbox
 - Statistics Toolbox
 - Parallel Computing Toolbox
2. **Install missing toolboxes (if MATLAB is already installed).** Use the Add-On Explorer in the **Add-Ons** menu on the Home tab. You will need to restart MATLAB after each toolbox installs.
 3. **Install CMEanalysis.** Go to github.com/DanuserLab/cmeAnalysis, click the green **Code** button, and download the ZIP. Unzip the folder somewhere stable on your computer (e.g. `Documents/cmeAnalysis-master`). **Remember the path where you put it** — you will use it in Step 2.

3.5 DLS data

You need a DLS measurement of your liposome stock from a Malvern Zetasizer. Export the size distribution into an Excel spreadsheet.

Important: DLS data formatting. Your Excel spreadsheet must match the layout of the example file in the `test_data/` folder of this repository. Open `test_data/dls/example_dls.xlsx` in Excel before formatting your own — copy the exact column layout. Specifically, the first column must contain the section header **X Number** followed by diameter bins, and subsequent columns hold the percentage data for each measurement record.

The script **only uses the number distribution** (X Number section). Other section headers like **X Intensity** and **X Volume** are ignored, but they should still be present so the file matches the standard Zetasizer export format.

4 Download the Code and Set Up

Do these steps once. After this you only need to activate the virtual environment (see §4.4) each time you open a new terminal.

4.1 Get the repository

Option A: with git (if installed):

```
git clone https://github.com/slele225/liposome-curvature-assay.git
```

Option B: download a ZIP. Go to github.com/slele225/liposome-curvature-assay, click the green **Code** button, then **Download ZIP**. Double-click to unzip. You should now have a folder called `liposome-curvature-assay`.

Move it somewhere you'll remember. We recommend putting it on your Desktop. The rest of this protocol assumes that location.

4.2 Open a terminal in the repo folder

macOS / Linux:

```
cd ~/Desktop/liposome-curvature-assay
```

Windows PowerShell:

```
cd $HOME\Desktop\liposome-curvature-assay
```

Tip. You can also type `cd` followed by a space, then drag and drop the folder from Finder/File Explorer into the terminal window. The full path will auto-fill.

4.3 Create a virtual environment

This creates a private space for the Python packages so they don't interfere with anything else on your computer.

```
uv venv
```

You should see a message like **Creating virtual environment at: .venv**. A new hidden folder called `.venv` now exists inside the repo.

4.4 Activate the virtual environment

macOS / Linux:

```
source .venv/bin/activate
```

Windows PowerShell:

```
.venv\Scripts\Activate
```

How to tell it worked: your terminal prompt should now start with `(.venv)`. For example:

```
(.venv) yourname@computer ~ %
```

Important. You need to activate the venv **every time** you open a new terminal window. If you close the terminal and come back later, repeat the `cd` step *and* the activation step above.

4.5 Install the Python packages

```
uv pip install -r requirements.txt
```

This downloads numpy, matplotlib, tifffile, h5py, openpyxl, pandas, and scipy. It takes about a minute the first time, much faster on later installs.

4.6 Verify everything works

```
python plot_curvature.py --help
```

You should see a help message listing all the options for that script. If you get an error, check that `(.venv)` appears at the start of your terminal prompt. If not, re-activate the venv (§4.4).

4.7 Build the native detection backend (recommended)

This compiles `cme_detect`, the native spot detector used in Step 2 (Option A). Do it once; repeat only after updating the code.

Windows PowerShell (from the repo folder):

```
cd native_detection
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
```



```
cmake --build build --config Release
cd ..
```

The first `cmake` command takes about half a minute, the second a few minutes. The program is now at `native_detection\build\Release\cme_detect.exe`. Check it:

```
.\native_detection\build\Release\cme_detect.exe --help
```

macOS / Linux (uses the system GSL and libtiff):

```
cd native_detection
cmake -S . -B build -DCME_USE_SYSTEM_LIBS=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build
cd ..
./native_detection/build/cme_detect --help
```

No build tools? You can skip this and use the MATLAB route in Step 2 (Option B) instead. Nothing else in the pipeline depends on the native backend.

5 Folder Structure: Where to Put Your Files

The repo has two main folders for organizing your work. All your data and outputs go inside the repo folder. Here is the structure (only the most important files are shown):

```
liposome-curvature-assay/
|-- data/                      <-- YOUR DATA GOES HERE
|   |-- dls_data.xlsx          <-- DLS Excel export
|   '-- experiment_name/      <-- one subfolder per experiment
|       '-- 488nm_5.0pct_580V_561nm_3.2pct_500V/
|           |-- cell1/
|               |-- ch1/        <-- lipid channel (this example)
|               '-- ch2/        <-- protein channel (this example)
|                   '-- filtered_puncta_A_values.txt
|-- figures/                   <-- PLOTS SAVED HERE
|-- test_data/                 <-- example DLS file (reference format)
|-- native_detection/          <-- native C++ spot detector (cme_detect)
|-- prepare_input.py
|-- analyze_cpp.py             <-- Step 3 for the native route
|-- analyze_matlab.py          <-- Step 3 for the MATLAB route
|-- dls_calibration.py
|-- plot_curvature.py
|-- plot_histograms.py
|-- plot_overlay.py
|-- plot_dls.py
|-- plot_dls_comparison.py
|-- requirements.txt
'-- README.md
```

`data/` — Put your raw TIFF images in a subfolder here (e.g. `data/march_3_experiment/`). The

`prepare_input.py` script will create the detection-ready folder structure inside `data/`. Also put your DLS `.xlsx` file here.

`figures/` — This is where your final plots get saved. The plotting scripts create this folder automatically when you use `--save-dir figures/`.

`test_data/` — Reference data, including an example DLS file showing the required spreadsheet format.

Note. The `data/` and `figures/` folders are gitignored, meaning their contents will not be uploaded if you use git. Your microscopy data stays on your computer only.

6 Step 1: Prepare Images

What this does. Takes your raw multi-frame TIFF files from the microscope, splits them into separate per-channel folders, and organizes them into the structure that the detection step (native `cme_detect` or MATLAB CMEanalysis) expects. Files are automatically grouped by laser configuration. Olympus FV3000 metadata is parsed for active lasers (any laser with transmissivity > 0), each channel's excitation laser, and each channel's PMT voltage. The output folder name encodes all three, e.g. `488nm_5.0pct_580V_561nm_3.2pct_500V/`. Lambda-phase channels are skipped. Channel folders `ch1`, `ch2`, etc. follow wavelength-ascending order (so `ch1` is always the lowest active wavelength).

Before you start

- Put all your raw `.tif` files from one experiment into a subfolder inside `data/`. For example: `data/march_3_experiment/`.

Run a dry-run first

It's strongly recommended to first run with `--dry-run` to see what folder structure the script will create, without actually writing any files. Use this to verify that the metadata parsing matches what you actually acquired.

macOS / Linux:

```
python prepare_input.py \  
  --input data/march_3_experiment \  
  --output data/march_3_experiment_matlab \  
  --crop 1 \  
  --dry-run
```

Windows PowerShell:

```
python prepare_input.py '  
  --input data\march_3_experiment '  
  --output data\march_3_experiment_matlab '  
  --crop 1 '  
  --dry-run
```

The script prints, for each TIFF, the active lasers and their voltages, and the planned folder name.

Read this carefully. If anything looks wrong, fix it before running for real (most often, the issue is metadata interpretation rather than the script).

Run for real

If the dry-run output looks correct, repeat the same command without `--dry-run`:

macOS / Linux:

```
python prepare_input.py \  
  --input data/march_3_experiment \  
  --output data/march_3_experiment_matlab \  
  --crop 1
```

Windows PowerShell:

```
python prepare_input.py '  
  --input data\march_3_experiment '  
  --output data\march_3_experiment_matlab '  
  --crop 1
```

All flags

Flag	Meaning
<code>--input</code>	Folder containing raw <code>.tif</code> files.
<code>--output</code>	Where to save the split channels (inside <code>data/</code>).
<code>--frames</code>	Comma-separated frame indices, one per active channel in wavelength-ascending order (e.g. <code>0,1,2</code>). Optional — defaults to <code>0,1,...,N-1</code> per TIFF based on its active channel count.
<code>--crop</code>	Center crop divisor. <code>1</code> = no crop, <code>2</code> = center quarter. Use <code>crop > 1</code> if your image has dark borders or you want to focus on the central region.
<code>--dry-run</code>	Parse metadata and print what would be created, without writing files.

Expected output

A new folder `data/march_3_experiment_matlab/` containing subfolders for each laser configuration, cell, and channel. For example:

```
data/march_3_experiment_matlab/  
  '-- 488nm_5.0pct_580V_561nm_3.2pct_500V/  
    |-- cell1/  
      |-- ch1/      <-- 488 nm (lipid in this example)  
      '-- ch2/      <-- 561 nm (protein in this example)  
    '-- cell2/  
      |-- ch1/  
      '-- ch2/
```

Which folder is which? Each condition folder also gets a `channel_map.json` recording, for every `ch<i>` folder, the physical acquisition channel it holds: laser wavelength, transmissivity, detector, PMT voltage and dye name. Channel folders are ordered by *wavelength*, not by role — so the lipid channel is **not always** `ch1`. Check `channel_map.json` (or the recorded wavelength/voltage metadata) to identify which folder holds your lipid channel; you will need this in Step 2 to pick the master channel.

7 Step 2: Run Spot Detection

What this does. Fits 2D Gaussians to every sub-diffraction- limit fluorescent spot in each channel image. This identifies individual liposomes and measures their amplitudes (brightness), backgrounds, and a quality flag.

There are two ways to run this step. Both implement the same CMEanalysis detection algorithm (Danuser Lab), and Step 3 filters their results identically:

- **Option A (recommended): native `cme_detect` backend.** A C++ program included in this repository (`native_detection/`). No MATLAB needed, runs in seconds to minutes, fully scriptable. It was regression-validated against the original MATLAB CMEanalysis and reproduced its detection and filter decisions on the tested data sets, with only negligible floating-point differences in rare numerically unstable fits.
- **Option B (reference/fallback): MATLAB CMEanalysis.** The original implementation. Use it if you cannot build the native backend, or when you want to cross-check against the reference.

Option A (recommended): native detection with `cme_detect`

You need the executable built once (§4.7).

1. **Identify your lipid/master channel.** Open the `channel_map.json` that `prepare_input.py` wrote into the condition folder and check which physical acquisition channel (laser wavelength, detector voltage, dye) each `ch1/ch2` folder holds. The lipid channel is **not necessarily** `ch1` — either folder may hold it.
2. **Run the detector.** `--channels` lists the channel folders with the **master (lipid) channel first**, and `--master` names that same first channel. The example assumes `ch1` is the lipid channel; if `ch2` is, use `--channels ch2,ch1 --master ch2`.

Windows PowerShell:

```
& ".\native_detection\build\Release\cme_detect.exe" '  
  --input "data\march_3_experiment_matlab\488nm_5.0pct_580V_561nm_3.2pct_500V" '  
  --channels ch1,ch2 '  
  --master ch1 '  
  --seed 1 '  
  --threads 8
```

macOS / Linux (after building with system libraries, §4.7):

```
./native_detection/build/cme_detect \
  --input data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V \
  --channels ch1,ch2 \
  --master ch1 \
  --seed 1 \
  --threads 8
```

--seed 1 is the reproducible equivalent of the `rng(1)` used in the MATLAB route (it seeds the data-derived PSF sigma estimation). --threads sets how many CPU cores are used; the results do not depend on it. The program refuses to run if --master is not the first entry of --channels.

3. **Write down the channel order you used.** Step 3 (`analyze_cpp.py`) must be given the same --channels list, master first; it also checks the order recorded in the output files and stops if they disagree.

Verifying success (native)

The master channel folder of every cell now has a `Detection` subfolder containing `detection_cpp.tsv` (plus a mask image), and the condition folder has a `cme_detect_output` folder with `detections_all.tsv`, `summary.tsv` and `sigma.tsv`. Only the master channel gets a `Detection` folder:

```
data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V/
|-- cme_detect_output/
|   |-- detections_all.tsv
|   |-- summary.tsv
|   '-- sigma.tsv
'-- cell1/
    |-- ch1/
    |   '-- Detection/
    |       |-- detection_cpp.tsv  <-- this file must exist
    |       '-- dmask.tif
    '-- ch2/
        '-- (TIFF image only, no Detection folder)
```

Option B (reference/fallback): MATLAB CMEanalysis

These instructions use **CMEanalysis**, the external MATLAB code base from the Danuser Lab (see Prerequisites). Skip this subsection if you used Option A.

Procedure (MATLAB)

1. **Open MATLAB and navigate to the CMEanalysis folder.** In the MATLAB command window, type the following two commands. **Replace the path** with the actual location where you unzipped CMEanalysis (whatever path you remembered from Prerequisites). The path below is just an example.

```
cd('/path/to/cmeAnalysis-master/software')
addpath(genpath(pwd))
```

2. **Load the image folder.** In the MATLAB command window, run:

```
data = loadConditionData();
```

A file selection window will open. Select the *condition folder* produced in Step 1 (the one named after the laser configuration, e.g. 488nm_5.0pct_580V_561nm_3.2pct_500V).

3. **Identify your lipid/master channel.** Before selecting anything, check which physical acquisition channel each `ch1/ch2` folder holds, using the `channel_map.json` written by `prepare_input.py` (and the wavelength/voltage encoded in the condition folder name). The lipid channel is **not necessarily** `ch1` — either folder may hold it, depending on the acquisition.
4. **Select channels.** The program will ask you to select the *first (master) channel*. Select the **lipid channel folder** you identified above — whichever of `ch1` or `ch2` it is. It will then ask for the remaining (slave) channels — select the other channel folder(s).

Write down the selection order. The order you select channels here is the order CME-analysis stores them in `frameInfo`, and Step 3 (`analyze_matlab.py`) must be given *exactly the same order* via `--channels`, with the master channel first.

5. **Enter fluorophore names.** The program will ask for a fluorescent marker for each channel. Type the fluorophore name matching that channel. Common ones include `texasred` for Texas Red and `egfp` for EGFP. If the program does not recognize the name, it will ask you to enter the emission wavelength in nm instead. (Note: with a plain `runDetection(data)` the PSF sigma is estimated from the image data itself — the marker name you enter here is not what determines the default sigma estimate.)
6. **Run the detection.** The default mode determines the Gaussian-approximated PSF standard deviation directly from your images. For reproducibility, seed MATLAB's random number generator immediately before running the detection:

```
rng(1);  
runDetection(data);
```

Setting the seed right before `runDetection` makes CMEanalysis's data-derived PSF sigma estimation reproducible: the sigma-estimation step performs Gaussian-mixture fitting and can otherwise return slightly different sigma estimates on repeated runs of the same data, which then propagates into the detection and the fitted amplitudes.

If you have already run a detection on this folder and want to redo it (e.g. after changing parameters):

```
rng(1);  
runDetection(data, 'Overwrite', true);
```

If you know what sigma value to use from separate experiments or calculations:

```
runDetection(data, 'Overwrite', true, 'Sigma', sigma_value);
```

Replace `sigma_value` with your number. Note that sigma is the standard deviation, not the variance.

7. **Wait for completion.** The code will show progress and take a few minutes. It reports the detected standard deviations for each channel across all images and shows graphs with this data — you can close them.

Verifying success (MATLAB)

When MATLAB finishes, verify that the master channel folder — the one you selected *first* in `loadConditionData` — now has a `Detection` subfolder. Note that **only the master/lipid channel gets a Detection folder** — the slave channel folder(s) will just contain the TIFF file. This is expected. In the example below the master was `ch1`:

```
data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V/  
  '-- cell1/  
    |-- ch1/  
      |-- Detection/  
        '-- detection_v2.mat    <-- this file must exist  
  '-- ch2/  
    '-- (TIFF image only, no Detection folder)
```

Note. The `detection_v2.mat` files contain a struct called `frameInfo` with fields `A` (amplitude), `c` (background), and `hval_Ar` (hypothesis test value). The next Python step reads these fields.

8 Step 3: Analyze Detection Output

What this does. Reads the detection files from Step 2, filters out noise and poorly-fit spots, and exports a single tab-separated text file with the kept amplitude values for every liposome that passed quality control.

Use the script that matches the Step 2 route you took. Both take the same flags, apply the identical filter (they share the same code) and write identical output files:

- **Option A (native `cme_detect`):** `analyze_cpp.py`, which reads the `detection_cpp.tsv` file in each cell's `<master>/Detection/` folder.
- **Option B (MATLAB):** `analyze_matlab.py`, which reads the `detection_v2.mat` file in each cell's `<master>/Detection/` folder.

Channel order matters

`--channels` must list the channel folders in the **exact analysis order** — the list you gave to `cme_detect --channels`, or the order you selected them in MATLAB `loadConditionData` (the order stored in `frameInfo`) — *not* in numerical folder order. The first entry is the master/source channel, so `--lipid-channel` must equal the **first** entry of `--channels`. The scripts check this and stop with an error if the order is inconsistent, because a mismatch would attach the detection

results to the wrong physical channel. `analyze_cpp.py` additionally compares `--channels` with the order recorded in the `cme_detect` output and stops on a mismatch.

- If `ch1` was the master and `ch2` the slave: `--channels ch1,ch2 --lipid-channel ch1`
- If `ch2` was the master and `ch1` the slave: `--channels ch2,ch1 --lipid-channel ch2`

Run the script (Option A, native route)

The commands below assume `ch1` was the master — adjust the two channel flags to match *your* recorded order.

macOS / Linux (two-channel: lipid + protein):

```
python analyze_cpp.py \  
  --input data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V \  
  --channels ch1,ch2 \  
  --lipid-channel ch1 \  
  --k-std 2.0 \  
  --output-name filtered_puncta_A_values.txt
```

Windows PowerShell:

```
python analyze_cpp.py '  
  --input data\march_3_experiment_matlab\488nm_5.0pct_580V_561nm_3.2pct_500V '  
  --channels ch1,ch2 '  
  --lipid-channel ch1 '  
  --k-std 2.0 '  
  --output-name filtered_puncta_A_values.txt
```

Run the script (Option B, MATLAB route)

Identical, with `analyze_matlab.py` in place of `analyze_cpp.py`:

macOS / Linux:

```
python analyze_matlab.py \  
  --input data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V \  
  --channels ch1,ch2 \  
  --lipid-channel ch1 \  
  --k-std 2.0 \  
  --output-name filtered_puncta_A_values.txt
```

Windows PowerShell:

```
python analyze_matlab.py '  
  --input data\march_3_experiment_matlab\488nm_5.0pct_580V_561nm_3.2pct_500V '  
  --channels ch1,ch2 '  
  --lipid-channel ch1 '  
  --k-std 2.0 '  
  --output-name filtered_puncta_A_values.txt
```


Lipid-only experiments

If you only have a lipid channel (no protein), use `--channels ch1` and the same channel for `--lipid-channel` (with `analyze_matlab.py` for the MATLAB route):

```
python analyze_cpp.py \  
  --input data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V \  
  --channels ch1 \  
  --lipid-channel ch1
```

All flags

Both scripts accept the same flags.

Flag	Meaning
<code>--input</code>	Condition folder (the laser-configuration folder from Step 1).
<code>--channels</code>	Channel folder names, comma-separated, in the exact analysis order : as given to <code>cme_detect --channels</code> , or as selected in MATLAB <code>loadConditionData</code> / stored in <code>frameInfo</code> . The first entry is the master/source channel. Use a single name (e.g. <code>ch1</code>) for lipid-only.
<code>--lipid-channel</code>	The lipid/master channel (used to compute the intensity threshold). Must equal the first entry of <code>--channels</code> .
<code>--k-std</code>	Threshold strictness. Keeps puncta where amplitude $> \mu_c + k \cdot \sigma_c$, with μ_c and σ_c the mean and standard deviation of the background. Default 2.0. Larger = stricter, fewer puncta.
<code>--output-name</code>	Output filename. Saved inside the <code>--input</code> folder. Default <code>filtered_puncta_A_values.txt</code> .

Expected output

The script prints a per-cell summary showing how many puncta were kept vs. how many were detected. At the end, it saves a tab-separated text file:

```
data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V/  
  filtered_puncta_A_values.txt
```

The file has one amplitude column per channel, in the same order as `--channels` (master/lipid first) — e.g. `source_image`, `A_ch1` (lipid amplitude), `A_ch2` (protein amplitude) if `ch1` was the master. Each row is one liposome. Open it in Excel to spot-check.

9 Step 4: DLS Calibration

What this does. Reads your DLS Zetasizer spreadsheet and your filtered puncta file from Step 3, then computes a single *conversion factor* that maps the square root of fluorescence amplitude to physical diameter in nanometers. The script does this by overlaying the fluorescence $\sqrt{A}$ distribution onto the DLS number-weighted size distribution and finding the scalar that makes them best match.

This is the standard SLiC calibration approach (Kunding 2008, Hatzakis 2009, Bhatia 2009, Zeno 2018, Johnson 2025). A simpler “ratio of means” factor is also reported as a sanity check; the overlay is more robust because it uses the full distribution shape.

Before you start

Confirm your DLS spreadsheet matches the example format in `test_data/dls/`. See the note in the Prerequisites section above.

Run the script

macOS / Linux:

```
python dls_calibration.py \  
  --dls-input data/dls_data.xlsx \  
  --fluor-input data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V/  
  filtered_puncta_A_values.txt \  
  --save-dir figures/
```

Windows PowerShell:

```
python dls_calibration.py '  
  --dls-input data\dls_data.xlsx '  
  --fluor-input data\march_3_experiment_matlab\488nm_5.0pct_580V_561nm_3.2pct_500V\  
  filtered_puncta_A_values.txt '  
  --save-dir figures\
```

Optional: bootstrap variance estimate

To estimate uncertainty in the conversion factor, add `--bootstrap N`. The script resamples the puncta with replacement N times and reports the coefficient of variation.

```
python dls_calibration.py \  
  --dls-input data/dls_data.xlsx \  
  --fluor-input data/.../filtered_puncta_A_values.txt \  
  --bootstrap 200 \  
  --save-dir figures/
```

All flags

Flag	Meaning
<code>--dls-input</code>	Path to the Zetasizer <code>.xlsx</code> export.
<code>--fluor-input</code>	Path to <code>filtered_puncta_A_values.txt</code> from Step 3.
<code>--bootstrap N</code>	Bootstrap resamples for variance estimate. Default 0 (off). Try 200–500 to estimate uncertainty.
<code>--bootstrap-k K</code>	Number of puncta to sample per bootstrap iteration. Default = same as total puncta.
<code>--lipid-col</code>	Column name for lipid amplitude. Default <code>A_ch1</code> .
<code>--save-dir</code>	Save the overlay plots here (optional).

Output: what to copy down

The script prints two key numbers near the bottom of its output. You will use one of them in Step 5:

- `python plot_curvature.py --conversion-factor X.XXXXXX ...` — copy this number for use as `--conversion-factor`.
- Equivalently, an implied mean diameter for use as `--dls-mean-diameter`. Either choice gives the same plot.

It also saves a figure `figures/dls_overlay_calibration.png` showing the DLS distribution (blue) and your fluorescence distribution (red, dashed) on the same axis. They should overlap. If they don't, your detection threshold (`--k-std` in Step 3) may need adjustment.

10 Step 5: Plot Curvature Sorting

What this does. Converts lipid amplitudes into physical liposome diameters using the calibration from Step 4, computes protein surface density for each liposome, and generates the final scatter plot of **protein density vs. liposome diameter**. This is the headline figure of the assay.

Run the script

macOS / Linux:

```
python plot_curvature.py \
  --input data/march_3_experiment_matlab/488nm_5.0pct_580V_561nm_3.2pct_500V/
  filtered_puncta_A_values.txt \
  --conversion-factor 1.234567 \
  --save-dir figures/
```

Windows PowerShell:

```
python plot_curvature.py '
  --input data\march_3_experiment_matlab\488nm_5.0pct_580V_561nm_3.2pct_500V\
  filtered_puncta_A_values.txt '
  --conversion-factor 1.234567 '
  --save-dir figures\
```

Replace 1.234567 with the conversion factor printed by Step 4.

Alternative: use a mean diameter directly

If you didn't run `dls_calibration.py`, you can use a DLS mean diameter directly (the script computes the ratio-of-means factor internally):

```
python plot_curvature.py \
  --input data/.../filtered_puncta_A_values.txt \
  --dls-mean-diameter 80.12 \
  --save-dir figures/
```

All flags

Flag	Meaning
<code>--input</code>	Filtered puncta file from Step 3. Accepts multiple files (one plot per file).
<code>--conversion-factor</code>	Maps $\sqrt{A}$ to diameter in nm. From <code>dls_calibration.py</code> . Provide this <i>or</i> <code>--dls-mean-diameter</code> , not both.
<code>--dls-mean-diameter</code>	Alternative: mean DLS diameter in nm. Internally uses ratio-of-means.
<code>--lipid-col</code>	Column for lipid amplitude. Default <code>A_ch1</code> .
<code>--protein-col</code>	Column for protein amplitude. Default <code>A_ch2</code> .
<code>--bin-width</code>	Diameter bin width in nm for the averaged curve. Default 0.5. Larger = smoother.
<code>--diameter-cutoff</code>	Exclude puncta with diameter above this value (nm). Useful for filtering sparse large-diameter tail (optional).
<code>--y-pad</code>	Y-axis padding factor around bin means. Default 0.3 (30%). Lower values like 0.1 zoom in tighter on the trend.
<code>--save-dir</code>	Output directory for figures.

Expected output

A PNG saved in `figures/`: `488nm_5.0pct_580V_561nm_3.2pct_500V__protein_density_vs_diameter.png`.

Interpreting the plot

- **X-axis:** liposome diameter (nm). Smaller = higher curvature.
- **Y-axis:** protein surface density (amplitude per nm²). Higher = more protein per unit membrane area.
- Light dots are individual liposomes; larger dots are bin-averaged means (the trend line you care about).

What the trend tells you:

- Bin averages going **up** toward smaller diameters $\Rightarrow$ your protein preferentially binds high-curvature membranes (positive curvature sensing).
- Flat line $\Rightarrow$ no curvature preference.
- Bin averages going **down** toward smaller diameters $\Rightarrow$ negative curvature sensing.

11 Step 6 (Optional): Plot Histograms

A useful sanity-check tool. Plots amplitude histograms for each channel and an estimated diameter distribution. Works for both two-channel and lipid-only experiments. Most useful when you want to verify the shape of your data before making the curvature plot.

Two-channel

```
python plot_histograms.py \  
    --input data/.../filtered_puncta_A_values.txt \  
    --lipid-col A_ch1 \  
    --protein-col A_ch2
```

```
--protein-col A_ch2 \
--conversion-factor 1.234567 \
--save-dir figures/
```

Lipid-only (omit --protein-col)

```
python plot_histograms.py \
--input data/.../filtered_puncta_A_values.txt \
--lipid-col A_ch1 \
--conversion-factor 1.234567 \
--save-dir figures/
```

All flags

Flag	Meaning
--input	Filtered puncta file from Step 3.
--lipid-col	Lipid amplitude column. Default A_ch1.
--protein-col	Protein amplitude column. <i>Omit for lipid-only.</i>
--conversion-factor	From <code>dls_calibration.py</code> . Omit to skip the diameter plot.
--bins	Number of histogram bins. Default 80.
--transform	Amplitude transformation: raw (default), sqrt , or log_sqrt .
--save-dir	Output directory for figures.

12 Step 7 (Optional): Normalized Overlay Across Experiments

If you have multiple experiments (different proteins, conditions, replicates), overlay their normalized curvature-sorting curves on one plot. By default each curve is normalized so its rightmost (largest-diameter) bin equals 1, so the y-axis reads as *fold-enrichment at high curvature relative to flat membranes* — the same convention as Bhatia (2009) and Zeno (2018).

Run the script

Each input is given as `path:conversion_factor` (separated by a colon). Run `dls_calibration.py` *separately for each experiment* to get its own conversion factor.

macOS / Linux:

```
python plot_overlay.py \
--input data/cond1/filtered.txt:1.234 data/cond2/filtered.txt:1.567 \
--labels "WT protein" "Mutant K58A" \
--save-dir figures/
```

Windows PowerShell:

```
python plot_overlay.py '
--input data\cond1\filtered.txt:1.234 data\cond2\filtered.txt:1.567 '
--labels "WT protein" "Mutant K58A" '
--save-dir figures\
```

All flags

Flag	Meaning
--input	Files with conversion factors, as <code>file.txt:factor</code> pairs.
--labels	Custom legend labels (default = parent folder name of each file).
--lipid-col	Lipid column. Default <code>A_ch1</code> .
--protein-col	Protein column. Default <code>A_ch2</code> .
--bin-width	Diameter bin width in nm. Default 0.5.
--normalize-to	How to normalize curves: <code>rightmost</code> (default, matches Bhatia/Zeno), <code>leftmost</code> (for negative curvature sensors), <code>minimum</code> (always ≥ 1), or <code>none</code> .
--diameter-cutoff	Exclude puncta above this diameter (nm).
--y-pad	Y-axis padding factor. Default 0.3.
--output-name	Output filename. Default <code>normalized_curvature_overlay.png</code> .
--save-dir	Output directory.

13 Step 8 (Optional): DLS Distribution Plot

Quick standalone visualization of your DLS distribution.

```
python plot_dls.py data/dls_data.xlsx
python plot_dls.py data/dls_data.xlsx --log
python plot_dls.py data/dls_data.xlsx --distribution intensity
python plot_dls.py data/dls_data.xlsx --zoom-pct 100
```

Flag	Meaning
input	Path to Zetasizer <code>.xlsx</code> (positional).
--distribution	<code>number</code> (default) or <code>intensity</code> .
--log	Plot $\log(\text{diameter})$ instead of raw.
--zoom-pct	Percent of data to show (default 95).

14 Step 9 (Optional): DLS vs Fluorescence Comparison

Side-by-side comparison of the DLS distribution with the fluorescence $\sqrt{A}$ distributions for one or more channels. One panel per channel. Use this to verify that your detected liposome population matches the DLS measurement.

```
python plot_dls_comparison.py \
  --dls-input data/dls_data.xlsx \
  --fluor-input data/.../filtered_puncta_A_values.txt \
  --channels 0 Lipid 1 EGFP \
  --zoom-pct 95 \
  --bins 200 \
  --save-dir figures/
```

Flag	Meaning
<code>--dls-input</code>	DLS <code>.xlsx</code> file.
<code>--fluor-input</code>	Filtered puncta file.
<code>--channels</code>	Index/label pairs: 0 Lipid 1 EGFP. Index 0 = A_ch1, 1 = A_ch2.
<code>--dls-distribution</code>	number (default) or intensity for the DLS panel.
<code>--bins</code>	Bins for the fluorescence histogram (default 100).
<code>--zoom-pct</code>	Percent of data to show (default 100).
<code>--save-dir</code>	Output directory.

15 Quick Reference: Every-Time Checklist

After the initial setup is done, here's what you do each time you have new data:

1. Open a terminal.
2. Navigate to the repo:
 - Mac: `cd ~/Desktop/liposome-curvature-assay`
 - Windows: `cd $HOME\Desktop\liposome-curvature-assay`
3. Activate the venv:
 - Mac: `source .venv/bin/activate`
 - Windows: `.venv\Scripts\Activate`
4. Copy your raw TIFFs into a new subfolder inside `data/`.
5. Run `prepare_input.py` (Step 1) — with `--dry-run` first to verify.
6. Run spot detection (Step 2) — `cme_detect` (recommended) or MATLAB CMEanalysis (fallback).
7. Run `analyze_cpp.py` (native route) or `analyze_matlab.py` (MATLAB route) (Step 3).
8. Run `dls_calibration.py` (Step 4) — copy down the conversion factor.
9. Run `plot_curvature.py` (Step 5) with that factor and `--save-dir figures/`.
10. Find your plot in the `figures/` folder.

16 Troubleshooting

This section covers errors you are likely to encounter. Most issues trace back to one of three things: the virtual environment is not active, a path is wrong, or the data file isn't where the script expects.

16.1 Setup errors

“python is not recognized as an internal or external command” (Windows). Python wasn't added to `PATH` during install. Either reinstall Python and check “Add python.exe to PATH,” or use the full path to `python.exe`.

“python3 is not recognized” (Windows). On Windows, use `python` (not `python3`). The `python3` convention is Mac/Linux only.

“command not found: python” (Mac). Try python3. On some Macs, python isn’t linked but python3 is. If neither works, install Python (see Prerequisites).

“command not found: uv.” Close and reopen your terminal after installing uv. If still not found, reinstall uv with the commands in Prerequisites.

“No module named tiff file/numpy/pandas/...” Your virtual environment is not active. Run `source .venv/bin/activate` (Mac) or `.venv\Scripts\Activate` (Windows). Look for `(.venv)` at the start of your prompt.

Activation script can’t be run on Windows (PowerShell execution policy error). Run this once, then try activating again:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

“Permission denied” on Mac. For the uv install script: prepend `sudo`. For `.venv`: delete the `.venv` folder and recreate it with `uv venv`.

16.2 File and path errors

“FileNotFoundError: [...] filtered_puncta_A_values.txt.” Either the path is wrong, or Step 3 didn’t write the file. Paste your path into Finder/File Explorer to confirm it exists. If the path contains spaces, wrap it in double quotes.

“No .tif files found.” `--input` points to the wrong folder. It should be the folder containing your TIFFs directly, not a parent or sibling.

“missing detection file.” Detection hasn’t been run on that cell, or it failed. Open the cell folder in Finder/Explorer and check whether the master channel folder (the channel you put first in `cme_detect --channels`, or selected first in MATLAB, e.g. `ch1`) has a `Detection/` subfolder containing `detection_cpp.tsv` (native route) or `detection_v2.mat` (MATLAB route). Also make sure you are running the script matching your route (`analyze_cpp.py` vs. `analyze_matlab.py`).

Path with spaces causes an error. Wrap the entire path in double quotes:

```
python prepare_input.py --input "data/march 3 experiment" ...
```

16.3 Pipeline-stage errors

Step 1: “Frames out of range.” The frame indices in `--frames` don’t match what’s in the TIFF. Open one TIFF in ImageJ and check how many frames it has. Frame indices start at 0.

Setup: “cmake is not recognized” (Windows). The Visual Studio Build Tools are missing the “Desktop development with C++” workload, or the terminal was opened before installing. Re-run the installer, then open a new terminal — or use **Developer PowerShell for VS 2022** from the Start menu. If you cannot install build tools, use the MATLAB route (Step 2, Option B).

Step 2: “--master (...) must equal the first entry of --channels.” `cme_detect` refuses to reorder channels. Put the lipid/master channel first in `--channels` and name the same channel in `--master`, e.g. `--channels ch2,ch1 --master ch2`.

Step 2: `cme_detect` says “Channel directory not found” or “No movies found.” `--input` must be the condition folder (the one named after the laser configuration) that contains the `cell1/`,

cell12/, ... folders, each with the channel subfolders named in `--channels`.

Step 2: MATLAB detection produced no Detection/ folder. Make sure you selected your lipid/master channel *first* in `loadConditionData()`. Only the master (first- selected) channel gets a Detection/ folder.

Step 3: “--lipid-channel ... is not the FIRST entry in --channels.” `--channels` must repeat the exact order you gave to `cme_detect` (or selected in MATLAB), master first, and `--lipid-channel` must equal that first entry. E.g. if `ch2` was the master and `ch1` the slave, use `--channels ch2,ch1 --lipid-channel ch2`.

Step 3: “channel order mismatch” from `analyze_cpp.py`. The `--channels` list differs from the one `cme_detect` was run with (the output files record it). Pass exactly the same list, master first.

Step 3: “No puncta passed the filter.” The threshold (`--k-std`) is too strict, or signal is too low. Try `--k-std 1.5` or `--k-std 1.0`.

Step 4: “Could not find ‘X Number’ section in spreadsheet.” Your DLS Excel file is not in the expected format. Open `test_data/dls/example_dls.xlsx` and copy that exact layout. The first column must contain the section header `X Number` followed by diameter bins.

Step 4: implied mean diameter is way off (e.g. < 10 nm or > 1000 nm). The conversion likely failed because of a Step 3 issue. Run `plot_histograms.py` to look at your fluorescence distribution — it should be roughly unimodal and positive. If it looks bad, revisit your `--k-std` threshold in Step 3.

Step 5: x-axis range doesn’t match DLS. You’re applying the conversion factor wrong, or the implied conversion factor is bad. Make sure you copied the exact number printed by `dls_calibration.py` into `--conversion-factor`, not a derived value.

16.4 Plot quality issues

Plot looks flat / no trend visible. The y-axis is being auto-scaled to outliers. Add `--y-pad 0.1` to zoom in on the trend, or use `--diameter-cutoff 120` to drop the noisy large-diameter tail.

Curve is very noisy. Increase `--bin-width` (try 1.0 or 2.0). Larger bins average over more puncta and smooth out noise.

Plot’s diameter range is unreasonable. See “Step 5: x-axis range doesn’t match DLS” above.

16.5 General

I closed my terminal and now nothing works. You need to `cd` back to the repo folder and activate the venv again. See the Quick Reference checklist.

“UnicodeDecodeError” or weird text errors. Your DLS `.xlsx` may have been opened in Google Sheets and saved as `.csv`. Re-export from the Malvern software as a fresh `.xlsx`.

17 Common Pitfalls

A few extra things to watch out for, gathered from real users running this pipeline:

- **Forgetting to activate the venv.** The single most common error. Always check that your prompt starts with `(.venv)`.

- **Mixing up `--conversion-factor` and `--dls-mean-diameter`.** They are not the same number. The conversion factor is on the order of 1 to 10 (units of $\text{nm}/\sqrt{A}$); the mean diameter is in nanometers (typically 50–200 nm). If your plot's x-axis is way off, you probably swapped them.
- **Re-using an old conversion factor for new data.** Each liposome stock has its own DLS distribution and its own calibration. Always re-run `dls_calibration.py` for new data.
- **Comparing two experiments without separate calibrations.** For `plot_overlay.py`, each input file needs its own conversion factor. Don't reuse one factor across multiple experiments.
- **Saving the plot to the wrong folder.** If you forget `--save-dir figures/`, the plot saves in your current working directory. Always pass the flag.
- **Spaces in folder names without quotes.** Wrap any path with spaces in double quotes: `"data/march 3 experiment"`.
- **Editing files in Excel and saving as .csv.** Some DLS workflows tempt you to save as CSV. Don't — `dls_calibration.py` expects `.xlsx` with the Zetasizer section headers in column A.

18 Further Reading

- `README.md` — terse developer reference for every command.
- `DLS_Calibration_Notes.md` — theoretical background on the distribution-overlay calibration: why it works, how it relates to published SLiC protocols, what the reported statistics mean.
- Original assay references: Kunding et al. (2008) *Biophys J* 95:1176; Hatzakis et al. (2009) *Nat Chem Biol* 5:835; Bhatia et al. (2009) *EMBO J* 28:3303; Zeno et al. (2018) *Nat Commun* 9:4152; Johnson et al. (2025) *Commun Biol* 8:1179.